`Docker-02`

1. **Create a tomcat container on 8080 and deploy sample application in tomcat.**

Task 8 — Create a Tomcat Container and Deploy a Sample Application

Objective

To run a Tomcat server inside a Docker container, deploy a sample web application, and access it through a browser.

Step 1: Pull the Tomcat image

docker pull tomcat:9.0

Purpose: Downloads the Tomcat 9.0 Docker image from Docker Hub.

Why? We need the Tomcat image to create and run a Tomcat server inside a container.

Step 2: Create and run the Tomcat container

docker run -d --name tomcat-app -p 8080:8080 tomcat:9.0

Purpose: Creates and starts a container named `tomcat-app`.

- `-d` — Runs the container in the background.
- `--name tomcat-app` — Gives the container a name.
- `-p 8080:8080` — Maps port 8080 on the EC2 server to port 8080 inside the container.
- `tomcat:9.0` — Specifies the Tomcat image.

Step 3: Verify the container

docker ps

Purpose: Checks whether the Tomcat container is running.

Expected result: The `tomcat-app` container should show a status such as `Up`.

Step 4: Create a sample web application

docker exec tomcat-app sh -c 'mkdir -p /usr/local/tomcat/webapps/sample'

Purpose: Creates a directory named `sample` inside Tomcat's web applications folder.

Why? Tomcat uses this folder to host web applications.

Step 5: Create the sample HTML page

docker exec tomcat-app sh -c 'echo "<html><body><h1>Hello from Tomcat Docker!</h1> My sample application is running. </body></html>" > /usr/local/tomcat/webapps/sample/index.html'

Purpose: Creates an HTML page inside the sample application directory.

Why? This page helps us confirm that the application is accessible through Tomcat.

Step 6: Test the application

Run:

curl http://localhost:8080/sample/

Purpose: Checks whether Tomcat is serving the sample application.

Expected output includes:

<h1>Hello from Tomcat Docker!</h1>

Step 7: Access the application in a browser

Open:

`http://YOUR-EC2-PUBLIC-IP:8080/sample/`

Replace `YOUR-EC2-PUBLIC-IP` with your EC2 instance's public IP address. Ensure the EC2 security group allows inbound TCP traffic on port `8080` from your IP address.

How does this work in real time?

Tomcat Docker Image

Downloaded from Docker Hub

Tomcat Container

Runs the Tomcat server on port 8080

Sample Web Application

Contains index.html

Web Browser

Displays the sample web page

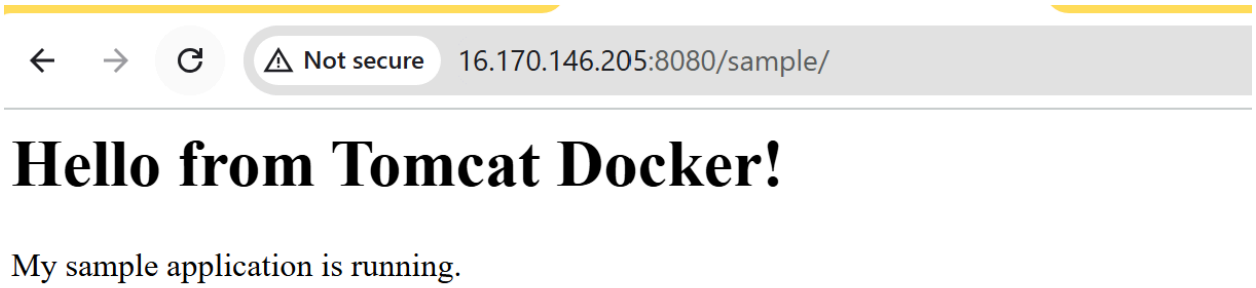
Real-time purpose

In real projects, developers package web applications and deploy them to application servers such as Tomcat. Docker helps run the server in an isolated, repeatable environment.

Conclusion: We pulled the Tomcat image, created a running container, deployed a sample HTML page, and tested the application through port 8080.

```
13-Oct-2026 11:55:59.507 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: --add-opens=java.base/java.util.concurrent=ALL-
13-Oct-2026 11:55:59.507 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: --add-opens=java.rmi/sun.rmi.transport=ALL-UNNA
13-Oct-2026 11:55:59.507 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.util.logging.config.file=/usr/local/tomc
ng.properties
13-Oct-2026 11:55:59.507 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.util.logging.manager=org.apache.juli.Cla
nager
13-Oct-2026 11:55:59.507 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djdk.tls.ephemeralDHKeySize=2048
13-Oct-2026 11:55:59.507 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.protocol.handler.pkgs=org.apache.catali
s
13-Oct-2026 11:55:59.508 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Dsun.io.useCanonCaches=false
13-Oct-2026 11:55:59.508 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Dorg.apache.catalina.security.SecurityListene
13-Oct-2026 11:55:59.511 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Dignore.endorsed.dirs=
13-Oct-2026 11:55:59.511 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Dcatalina.base=/usr/local/tomcat
13-Oct-2026 11:55:59.511 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Dcatalina.home=/usr/local/tomcat
13-Oct-2026 11:55:59.511 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.io.tmpdir=/usr/local/tomcat/temp
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.apache.tomcat.jni.Library in an unnamed module (file:/usr/local/tomcat/lib/tomcat-jni.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

13-Oct-2026 11:55:59.521 INFO [main] org.apache.catalina.core.AprLifecycleListener.lifecycleEvent Loaded Apache Tomcat Native library [1.3.9] using APR version
13-Oct-2026 11:55:59.522 INFO [main] org.apache.catalina.core.AprLifecycleListener.lifecycleEvent APR capabilities: IPv6 [true], sendfile [true], accept filters
om [true], UDS [true].
13-Oct-2026 11:55:59.522 INFO [main] org.apache.catalina.core.AprLifecycleListener.lifecycleEvent APR/OpenSSL configuration: useAprConnector [false], useOpenSSL
13-Oct-2026 11:55:59.532 INFO [main] org.apache.catalina.core.AprLifecycleListener.initializeSSL OpenSSL successfully initialized [OpenSSL 3.5.5 27 Jan 2026]
13-Oct-2026 11:55:59.853 INFO [main] org.apache.coyote.AbstractProtocol.init Initializing ProtocolHandler ["http-nio-8080"]
13-Oct-2026 11:55:59.875 INFO [main] org.apache.catalina.startup.Catalina.load Server initialization in [571] milliseconds
13-Oct-2026 11:55:59.916 INFO [main] org.apache.catalina.core.StandardService.startInternal Starting service [Catalina]
13-Oct-2026 11:55:59.917 INFO [main] org.apache.catalina.core.StandardEngine.startInternal Starting Servlet engine: [Apache Tomcat/9.0.123]
13-Oct-2026 11:55:59.925 INFO [main] org.apache.coyote.AbstractProtocol.start Starting ProtocolHandler ["http-nio-8080"]
13-Oct-2026 11:55:59.944 INFO [main] org.apache.catalina.startup.Catalina.start Server startup in [67] milliseconds
root@ip-172-31-23-83:/home/ubuntu# docker exec tomcat-app sh -c 'mkdir -p /usr/local/tomcat/webapps/sample'
root@ip-172-31-23-83:/home/ubuntu# docker exec tomcat-app sh -c 'echo "<html><body><h1>Hello from Tomcat Docker!</h1>docker exec tomcat-app sh -c 'echo "<html><b
 from Tomcat Docker!</h1><p>My sample application is running.</p></body></html>" > /usr/local/tomcat/webapps/sample/index.html'
root@ip-172-31-23-83:/home/ubuntu# curl http://localhost:8080/sample/
<html><body><h1>Hello from Tomcat Docker!</h1><p>My sample application is running.</p></body></html>
root@ip-172-31-23-83:/home/ubuntu# <h1>Hello from Tomcat Docker!</h1>
bash: syntax error near unexpected token `newline'
root@ip-172-31-23-83:/home/ubuntu# docker ps
CONTAINER ID   IMAGE        COMMAND             CREATED         STATUS         PORTS                                                  NAMES
a8e1f37a8f6e   tomcat:9.0   "catalina.sh run"   5 minutes ago   Up 5 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   tomcat-app
root@ip-172-31-23-83:/home/ubuntu# curl http://localhost:8080/sample/
<html><body><h1>Hello from Tomcat Docker!</h1><p>My sample application is running.</p></body></html>
root@ip-172-31-23-83:/home/ubuntu#
```



Task 2 — Create a Docker Volume and Deploy Tomcat on Port 8081

Objective

To create a Docker volume, attach it to a Tomcat container, and deploy a sample web application accessible through port 8081.

Step 1: Create a Docker volume

docker volume create tomcat-data

Purpose: Creates a named volume called `tomcat-data` to store application data outside the container's writable layer.

Step 2: Verify the volume

docker volume ls

Purpose: Displays Docker volumes and confirms that `tomcat-data` was created.

Step 3: Create a Tomcat container on port 8081

```
docker run -d \
  --name tomcat-8081 \
  -p 8081:8080 \
  -v tomcat-data:/usr/local/tomcat/webapps \
  tomcat:9.0
```

Purpose: Starts a Tomcat container, maps port 8081 on the EC2 server to port 8080 inside the container, and mounts the Docker volume.

Command explanation:

- `-d` — Runs the container in the background.
- `--name tomcat-8081` — Assigns a name to the container.
- `-p 8081:8080` — Maps host port 8081 to container port 8080.
- `-v tomcat-data:/usr/local/tomcat/webapps` — Mounts the volume at Tomcat's web applications directory.
- `tomcat:9.0` — Specifies the Tomcat image.

Step 4: Verify the container

docker ps

Purpose: Checks whether the container is running and verifies the port mapping.

Expected mapping:

0.0.0.0:8081->8080/tcp

Step 5: Create the application directory

docker exec tomcat-8081 sh -c 'mkdir -p /usr/local/tomcat/webapps/sample'

Purpose: Creates a directory for the sample web application inside the mounted volume.

Step 6: Create the sample HTML page

docker exec tomcat-8081 sh -c 'echo "<html><body><h1>Hello from Tomcat on Port 8081!</h1> Docker Volume is working. </body></html>" > /usr/local/tomcat/webapps/sample/index.html'

Purpose: Creates a web page that Tomcat can serve. The page is stored in the mounted volume.

Step 7: Test the application

curl http://localhost:8081/sample/

Purpose: Checks whether Tomcat serves the sample application successfully.

Expected output contains:

<h1>Hello from Tomcat on Port 8081!</h1>

You can also open the application in a browser:

`http://YOUR-EC2-PUBLIC-IP:8081/sample/`

Replace the placeholder with your EC2 public IP and ensure the security group allows inbound TCP traffic on port 8081.

Step 8: Verify the volume mount

docker inspect tomcat-8081 --format '{{json .Mounts}}'

Purpose: Confirms that the `tomcat-data` volume is attached to the container.

How does this work in real time?

Docker Volume: `tomcat-data`

Stores application files persistently

Tomcat Container: `tomcat-8081`

Reads and serves files from the volume

Browser — Port 8081

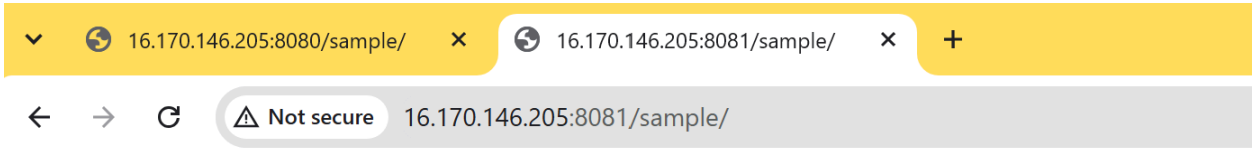
Displays the sample application

Real-time purpose

In real projects, Docker volumes are used to preserve application data independently of a container's lifecycle. Port mapping allows users to access a service running inside a container through a port on the host server.

Conclusion: We created the `tomcat-data` volume, attached it to a Tomcat container, deployed a sample HTML page, and tested the application on port 8081.

```
root@ip-172-31-23-83:/home/ubuntu# docker exec tomcat-app sh -c 'echo "<html><body><h1>Hello from Tomcat Docker!</h1>docker exec tomcat-app sh -c 'echo "<html><body><h1>Hel
 from Tomcat Docker!</h1><p>My sample application is running.</p></body></html>" > /usr/local/tomcat/webapps/sample/index.html'
root@ip-172-31-23-83:/home/ubuntu# curl http://localhost:8080/sample/
<html><body><h1>Hello from Tomcat Docker!</h1><p>My sample application is running.</p></body></html>
root@ip-172-31-23-83:/home/ubuntu# <h1>Hello from Tomcat Docker!</h1>
bash: syntax error near unexpected token `newline'
root@ip-172-31-23-83:/home/ubuntu# docker ps
CONTAINER ID   IMAGE        COMMAND             CREATED         STATUS         PORTS                                              NAMES
a8e1f37a8f6e   tomcat:9.0   "catalina.sh run"   5 minutes ago   Up 5 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   tomcat-app
root@ip-172-31-23-83:/home/ubuntu# curl http://localhost:8080/sample/
<html><body><h1>Hello from Tomcat Docker!</h1><p>My sample application is running.</p></body></html>
root@ip-172-31-23-83:/home/ubuntu# docker volume create tomcat-data
tomcat-data
root@ip-172-31-23-83:/home/ubuntu# docker volume ls
DRIVER    VOLUME NAME
local     jenkins_home
local     tomcat-data
root@ip-172-31-23-83:/home/ubuntu# docker run -d \
  --name tomcat-8081 \
  -p 8081:8080 \
  -v tomcat-data:/usr/local/tomcat/webapps \
  tomcat:9.0
ba1ed857c0cd013d87249b667fb4c327c0b50d3e316225e82eccf67e8f6b56a9
root@ip-172-31-23-83:/home/ubuntu# docker ps
CONTAINER ID   IMAGE        COMMAND             CREATED          STATUS          PORTS                                              NAMES
ba1ed857c0cd   tomcat:9.0   "catalina.sh run"   19 seconds ago   Up 18 seconds   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp   tomcat-8081
a8e1f37a8f6e   tomcat:9.0   "catalina.sh run"   14 minutes ago   Up 14 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   tomcat-app
root@ip-172-31-23-83:/home/ubuntu# docker exec tomcat-8081 sh -c 'mkdir -p /usr/local/tomcat/webapps/sample'                    docker exec tomcat-8081 sh -c 'mkdir -p /us
local/tomcat/webapps/sample'
root@ip-172-31-23-83:/home/ubuntu# docker exec tomcat-8081 sh -c 'echo "<html><body><h1>Hello from Tomcat on Port 8docker exec tomcat-8081 sh -c 'echo "<html><body><h1>Hell
from Tomcat on Port 8081!</h1><p>Docker Volume is working.</p></body></html>" > /usr/local/tomcat/webapps/sample/index.html'
root@ip-172-31-23-83:/home/ubuntu# curl http://localhost:8081/sample/
<html><body><h1>Hello from Tomcat on Port 8081!</h1><p>Docker Volume is working.</p></body></html>
root@ip-172-31-23-83:/home/ubuntu# dockdocker inspect tomcat-8081 --format '{{json .Mounts}}'e
[{"Type":"volume","Name":"tomcat-data","Source":"/var/lib/docker/volumes/tomcat-data/_data","Destination":"/usr/local/tomcat/webapps","Driver":"local","Mode":"z","RW":true,
ropagation":""}]
[{"Type":"volume","Name":"tomcat-data","Source":"/var/lib/docker/volumes/tomcat-data/_data","Destination":"/usr/local/tomcat/webapps","Driver":"local","Mode":"z","RW":true,
ropagation":""}]
error: no such object: inspect
root@ip-172-31-23-83:/home/ubuntu# docker inspect tomcat-8081 --format '{{json .Mounts}}'
[{"Type":"volume","Name":"tomcat-data","Source":"/var/lib/docker/volumes/tomcat-data/_data","Destination":"/usr/local/tomcat/webapps","Driver":"local","Mode":"z","RW":true,
ropagation":""}]
root@ip-172-31-23-83:/home/ubuntu#
```



Hello from Tomcat on Port 8081!

Docker Volume is working.

Task 3 — Limit the Nginx Container to 500 MB

Objective

To run an Nginx container with a maximum memory limit of 500 MB and verify that the web server is working.

Step 1: Run Nginx with a memory limit

docker run -d --name nginx-81 --memory=500m -p 81:80 nginx

Purpose: Creates and starts an Nginx container with a maximum memory allocation of 500 MB.

- `-d` — Runs in the background.
- `--name nginx-81` — Assigns the container name.
- `--memory=500m` — Sets the memory limit to 500 MB.
- `-p 81:80` — Maps host port 81 to container port 80.
- `nginx` — Specifies the Nginx image.

Step 2: Verify the container

docker ps

Purpose: Checks whether the Nginx container is running and confirms its port mapping.

Your output showed `nginx-81` as `Up`, with port mapping `81:80`.

Step 3: Verify the memory limit

docker inspect nginx-81 --format='Memory limit: {{.HostConfig.Memory}} bytes'

Purpose: Displays the configured memory limit.

Your output:

Memory limit: 524288000 bytes

This equals 500 MiB, which Docker specifies using `500m`.

Step 4: Test the Nginx web server

curl http://localhost:81

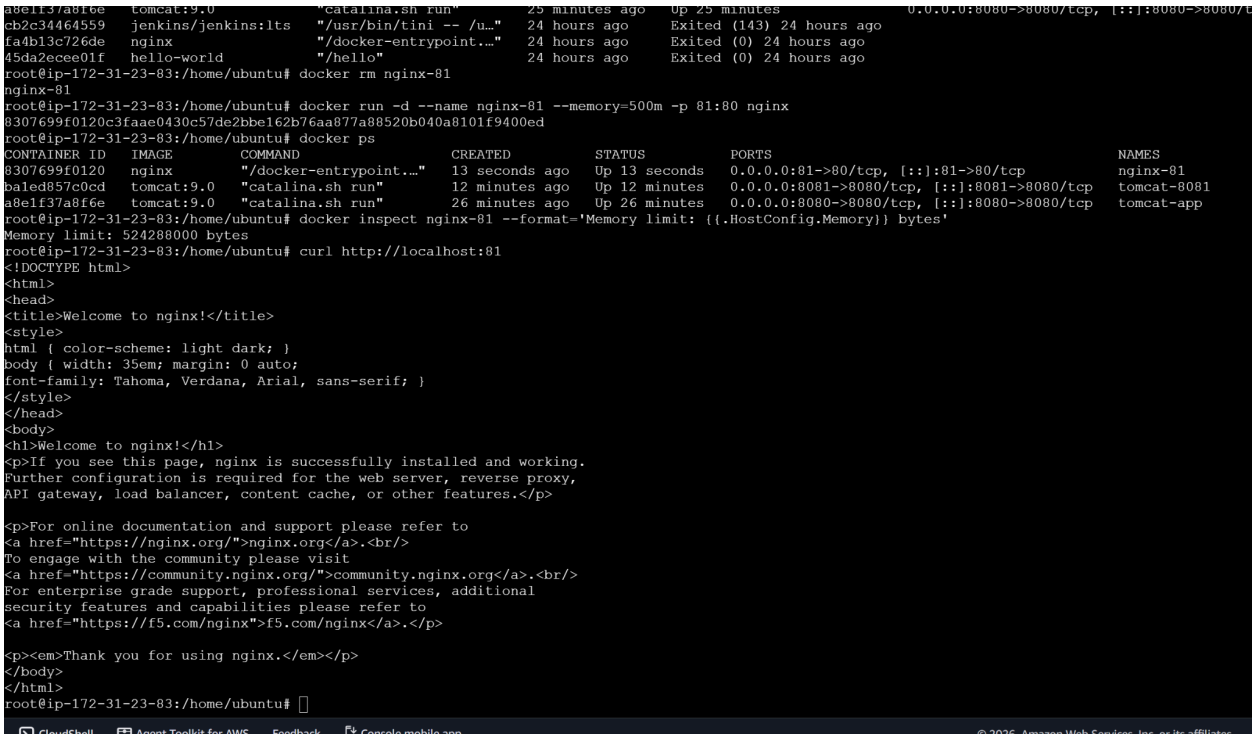
Purpose: Sends an HTTP request to Nginx to verify that it responds.

Result: You received the default “Welcome to nginx!” HTML page, confirming that the web server is working.

Real-time purpose

In real projects, memory limits help prevent a single container from consuming excessive RAM and affecting other applications on the same server.

Conclusion: We deployed Nginx on port 81, configured a 500 MB memory limit, verified the limit, and confirmed that the web server responds successfully.



Task 11 — Create a Dockerfile Using Amazon Linux and Nginx

Objective

To create a custom Docker image using `amazonlinux:latest`, install Nginx, copy an HTML file into the image, expose port 80, and start the Nginx server.

Step 1: Create a project directory

mkdir -p /home/ubuntu/nginx-docker
cd /home/ubuntu/nginx-docker

Purpose: Creates a separate directory to store the Dockerfile and HTML file.

Step 2: Create the HTML file

echo '<html><body><h1>Hello from my Docker image!</h1></body></html>' > index.html

Purpose: Creates a sample web page that Nginx will serve.

Step 3: Create the Dockerfile

nano Dockerfile

Dockerfile contents:

FROM amazonlinux:latest

LABEL maintainer="Your Name"

RUN dnf install -y nginx && dnf clean all

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

Purpose of each instruction:

Instruction	Purpose
`FROM amazonlinux:latest`	Selects Amazon Linux as the base image.
`LABEL maintainer="Your Name"`	Adds maintainer information.
`RUN dnf install -y nginx && dnf clean all`	Installs Nginx and cleans package metadata.
`COPY index.html /usr/share/nginx/html/index.html`	Copies the HTML page into Nginx's web directory.
`EXPOSE 80`	Documents the port Nginx listens on.
`CMD ["nginx", "-g", "daemon off;"]`	Starts Nginx in the foreground.

Replace `Your Name` with your name if you haven't already.

Step 4: Build the Docker image

docker build -t my-nginx-image .

Purpose: Builds a custom image from the Dockerfile.

Your output confirmed:

Successfully built 0a446cc0d2f3
Successfully tagged my-nginx-image:latest

Step 5: Verify the image

docker images

Purpose: Lists available Docker images and confirms that `my-nginx-image` exists.

Step 6: Run the container

docker run -d --name my-nginx-container -p 8082:80 my-nginx-image

Purpose: Starts the custom Nginx container.

- `-d` — Runs in the background.
- `--name my-nginx-container` — Names the container.
- `-p 8082:80` — Maps host port 8082 to container port 80.
- `my-nginx-image` — Uses the image created in Step 4.

Step 7: Test the web page

curl http://localhost:8082

Your output was:

<html><body><h1>Hello from my Docker image!</h1></body></html>

Purpose: Verifies that Nginx is running and serving your custom HTML page.

How does this work in real time?

Dockerfile + index.html

Defines how the image is built

Custom Docker Image

Amazon Linux + Nginx + HTML page

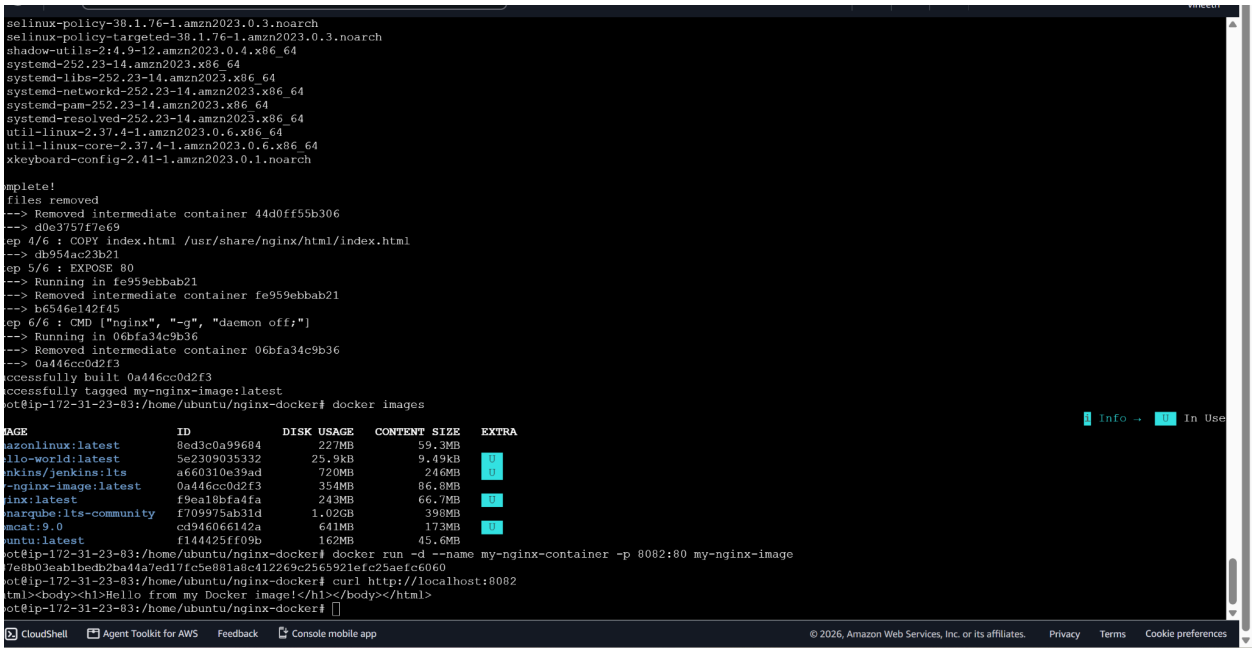
Running Container

Nginx serves the page on container port 80

Browser or HTTP Client

Accesses the application through host port 8082

Conclusion: We created a Dockerfile, built a custom Amazon Linux-based Nginx image, started a container on port 8082, and verified that our sample web page works.



Task 12 — Push a Docker Image to Docker Hub

Objective

To upload the custom Docker image `my-nginx-image` from the EC2 server to Docker Hub so it can be stored and reused on other servers.

Step 1: Create a Docker Hub repository

1. Open Docker Hub.
2. Sign in to your Docker Hub account.
3. Click Create repository.
4. Enter the repository name: `my-nginx-image`.
5. Select Public for this practice task and create the repository.

Purpose: Creates a remote location to store your Docker image.

Step 2: Log in to Docker Hub

docker login -u vineethgupta

Enter your Docker Hub access token when prompted for a password.

Purpose: Authenticates your Docker client so you can upload images to your account.

Note: If login fails due to a rate limit, wait before retrying. Do not share your access token.

Step 3: Tag the local image

docker tag my-nginx-image vineethgupta/my-nginx-image:latest

Purpose: Assigns your image a Docker Hub repository name.

- `my-nginx-image` — Your local image.
- `vineethgupta/my-nginx-image` — Your Docker Hub username and repository.
- `latest` — The image tag.

Step 4: Push the image

docker push vineethgupta/my-nginx-image:latest

Purpose: Uploads the image layers to Docker Hub.

Wait for the upload to finish. A successful push usually displays a digest and a `pushed` or `Layer already exists` status.

Step 5: Verify the image

1. Open Docker Hub.
2. Open your `my-nginx-image` repository.
3. Check that the `latest` tag is listed.

You can also check your local image tags using:

docker images

How it works in real time

EC2 Server

Custom Docker image built locally

Docker Login and Tag

Authenticate and identify the destination repository

Docker Hub

Stores the uploaded image for reuse

Conclusion: We build a Docker image, tag it with the Docker Hub username and repository, authenticate, and push it to Docker Hub for storage and reuse.

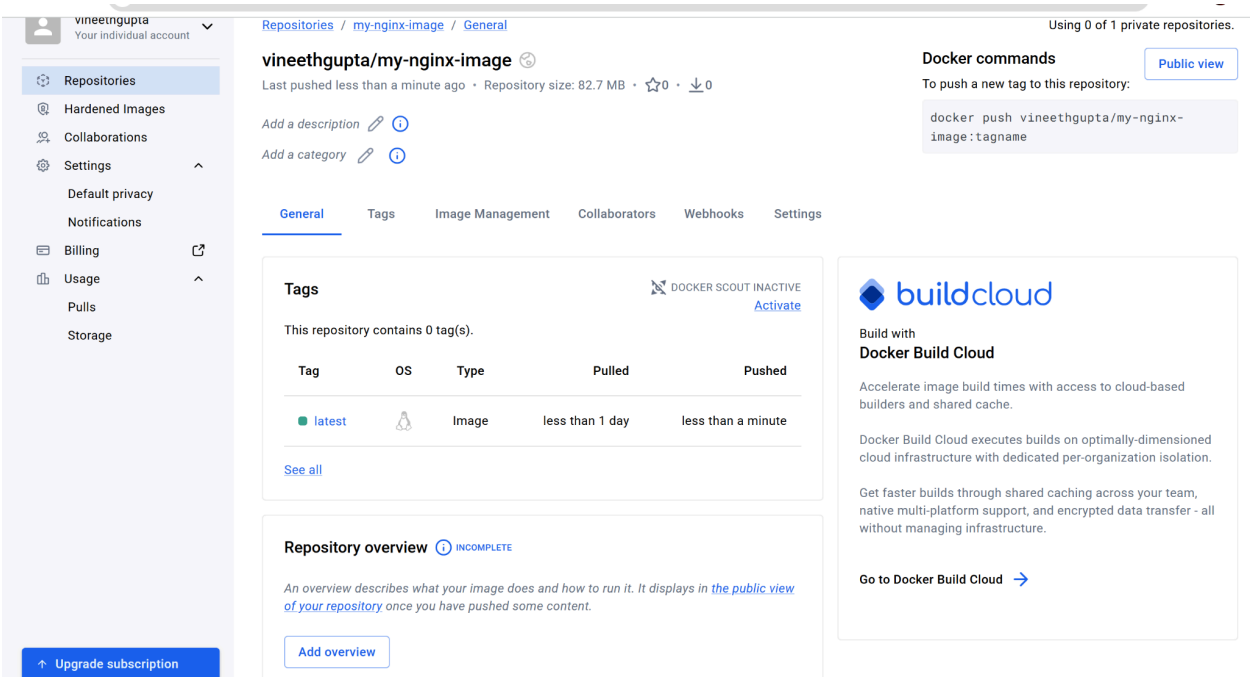
This might also interest you.

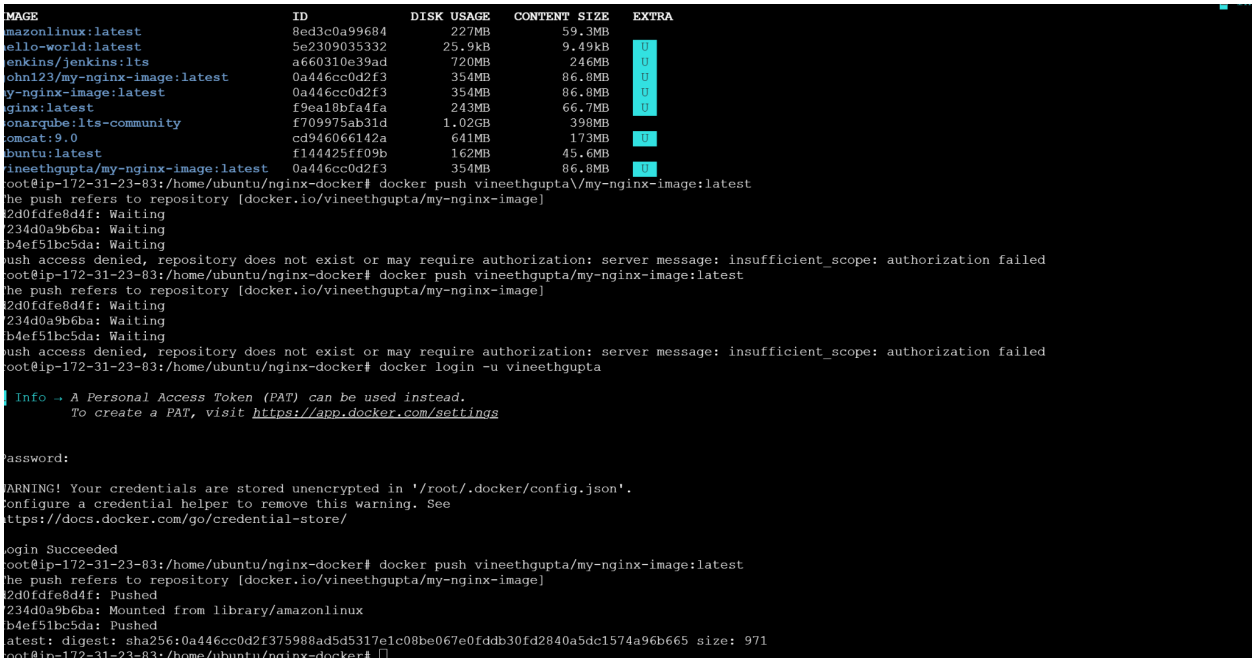
ManageEngine

Enterprise CIS Compliance Management

Assess CIS benchmark compliance and remediate gaps from one console.

Ad
Ad options





Task 6. Push a Docker Image to AWS ECR

Completed Successfully

Objective: Build a Docker image, authenticate with Amazon ECR, and push the image to a private ECR repository.

Step-by-step procedure

Step 1: Create an ECR repository

1. Open the AWS Console and search for Amazon ECR.
2. Select Private registry → Repositories.
3. Click Create repository.
4. Enter the repository name: `vineeth`
5. Select the region: `eu-north-1` (Europe — Stockholm).
6. Click Create repository.

You already completed this step.

Step 2: Configure IAM permissions

1. Open IAM → Roles → `pushrole`.
2. Ensure the role has permissions to authenticate and push images to ECR.

3. Attach the required ECR permissions to the role.

Your EC2 instance is already using this IAM role.

Step 3: Verify AWS authentication

Run this command on your EC2 terminal:

aws sts get-caller-identity

It confirmed your account ID is `063298174631` and the role is `pushrole`.

Step 4: Build a Docker image

You already created the local image named `my-nginx-image:latest`.

To build an image yourself in a future run, use:

docker build -t my-nginx-image .

This command assumes you have a valid `Dockerfile` in the current directory.

Step 5: Log in to Amazon ECR

Run:

aws ecr get-login-password --region eu-north-1 | docker login --username AWS --password-stdin 063298174631.dkr.ecr.eu-north-1.amazonaws.com

Expected output:

Login Succeeded

Step 6: Tag the Docker image for ECR

docker tag my-nginx-image:latest 063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth:latest

This gives the image the destination address of your ECR repository.

Step 7: Push the image to ECR

docker push 063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth:latest

Your successful output included:

latest: digest: sha256:0a446cc0d2f375988ad5d5317e1c08be067e0fddb30fd2840a5dc1574a96b665 size: 971

Step 8: Verify the image in AWS Console

1. Open Amazon ECR → Repositories → `vineeth`.
2. Select the Images tab.

3. Confirm that the `latest` image appears.

Your screenshot confirms the image is present and has a size of 86.76 MB.

Final result

Docker image pushed successfully

Amazon Elastic Container Registry (ECR)

AWS Region: `eu-north-1`

Repository: `vineeth`

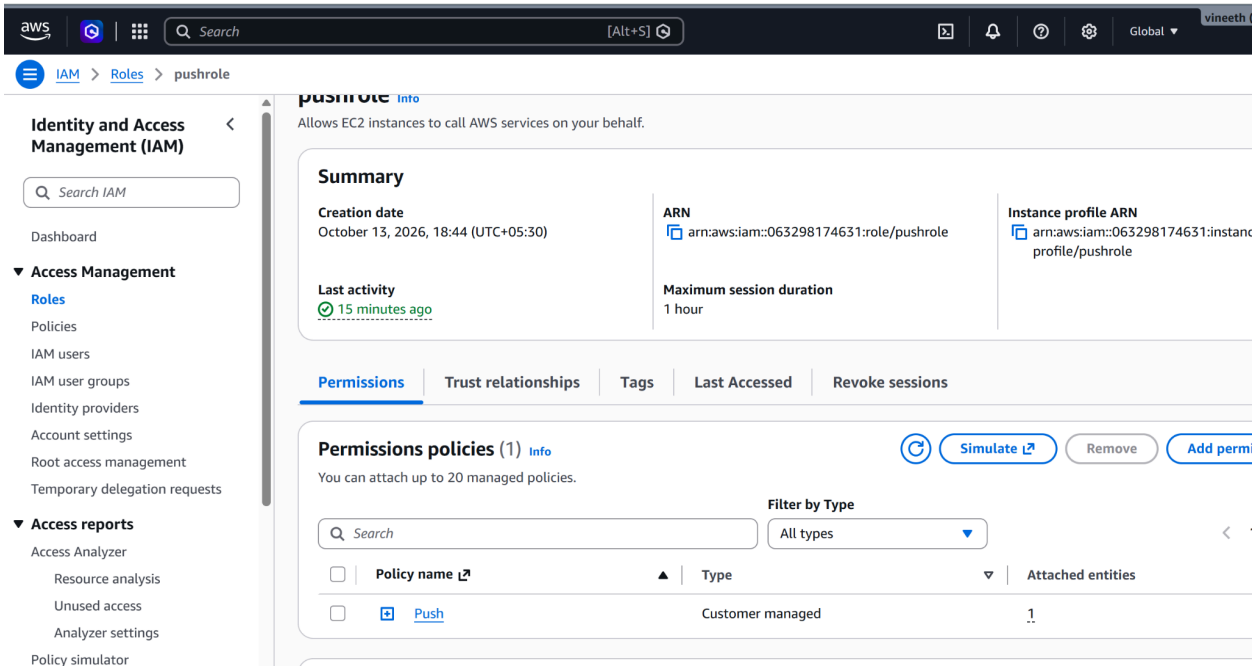
Image tag: `latest`

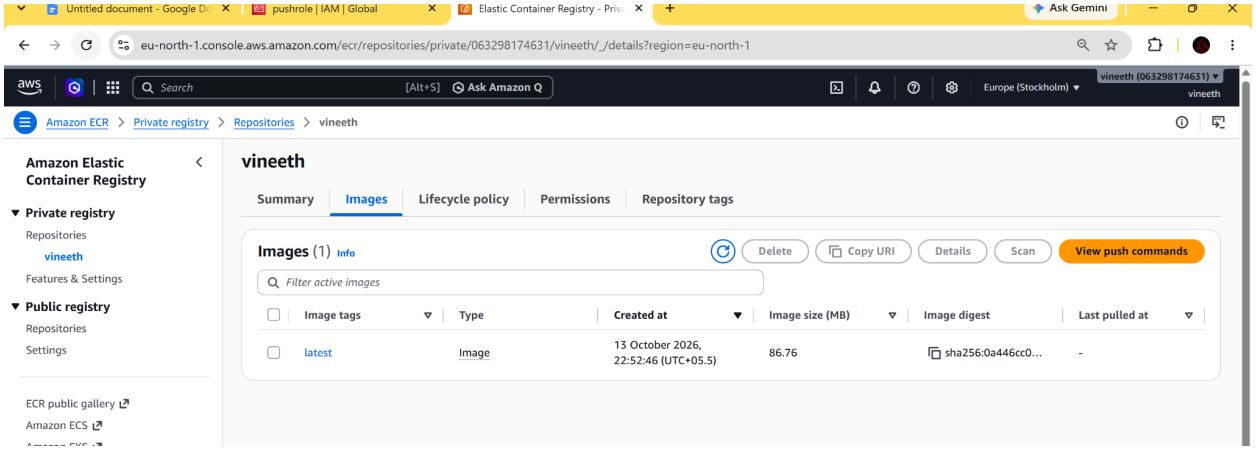
Image URI:

`063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth:latest`

Copy image URI

Conclusion: Successfully authenticated with AWS ECR, tagged the Docker image, pushed it to the private repository, and verified that the image is available in the AWS Console. The task “Push image to AWS ECR” is complete.






```
49 updates can be applied immediately.
2 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


*** System restart required ***
Last login: Tue Oct 13 17:07:33 2026 from 13.48.4.202
ubuntu@ip-172-31-23-83:~$ sudo su
root@ip-172-31-23-83:/home/ubuntu# aws sts get-caller-identity
{
    "UserId": "AROAQ5PG5P2TX4DZGPSBQ:i-020e2291f2c3332f9",
    "Account": "063298174631",
    "Arn": "arn:aws:sts::063298174631:assumed-role/pushrole/i-020e2291f2c3332f9"
}
root@ip-172-31-23-83:/home/ubuntu# aws ecr get-login-password --region eu-north-1 | docker login --username AWS --password-st

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-172-31-23-83:/home/ubuntu# docker push 063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth:latest
The push refers to repository [063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth]
d2d0fdfe8d4f: Layer already exists
7234d0a9b6ba: Layer already exists
fb4ef51bc5da: Layer already exists
unknown: unexpected status from HEAD request to https://063298174631.dkr.ecr.eu-north-1.amazonaws.com/v2/vineeth/manifests/la
root@ip-172-31-23-83:/home/ubuntu# docker push 063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth:latest
The push refers to repository [063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth]
d2d0fdfe8d4f: Layer already exists
7234d0a9b6ba: Layer already exists
fb4ef51bc5da: Layer already exists
latest: digest: sha256:0a446cc0d2f375988ad5d5317e1c08be067e0fddb30fd2840a5dc1574a96b665 size: 971
```